Data transmission

Types and methods of data transmission

Zain Merchant

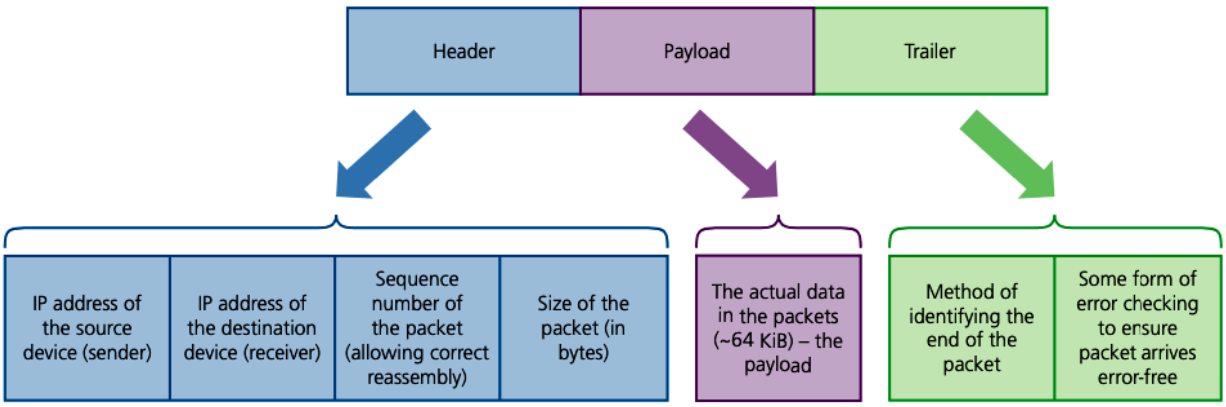
Types and methods of data transmission

Structure of a data packet

Zain Merchant

The amount of data that is stored in a file can be very large. If you tried to transmit this data from one device to another, all at once, this just wouldn't be possible or practical. The wires or radio waves used would simply not be able to accommodate sending such large amounts of data at a single time. Therefore, for the data in a file to be transmitted, it is broken down into very small units called packets.

Each packet of data contains three different sections:



The packet header contains a lot of important information about the data enclosed in the packet and its destination. The information it includes is the:

- destination address
- packet number
- originators address.

The destination address is normally an IP address. It is the IP address of the device where the data is being sent. Without this address, the hardware transmitting the data would not know where to send the data.

Each data packet in the file is given a packet number. The packets of data may not have all been sent in the correct order, this will depend on the type of transmission used. This means that when the destination device has received all the data packets, it can use the packet number to put them back into the correct order to recreate the file.

The originators address is also normally an IP address. It is the IP address of the device from which the data has been originally sent. So, if you send data to another device, the originator's address is the address of your device. This address isn't crucial to the transmission process, but it does help to trace where data has been sent from in situations such as illegal activity, or to simply request the original device to resend the data if an error is detected.

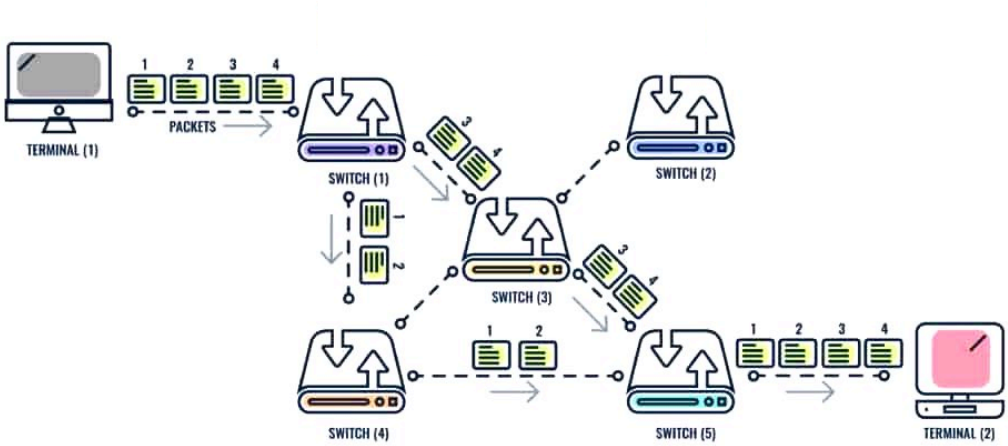
The payload of the data packet is the actual data from the file that you are sending. The data is broken up into many small units to be sent as the payload in each packet.

The trailer section of the packet is sometimes known as the footer. This contains two main pieces of information. The first is the marker to indicate it is the end of the packet and also the data for any error detection systems that are being used.

Packet switching

Zain Merchant

You have learnt that data is broken down into packets to be sent from one device to another. The process of transmitting these packets is called packet switching. In this process, each packet of data is sent individually from one device to another. There could be several pathways across a network that the packets could be transmitted. Each data packet could be sent along a different pathway. A router is the device that controls which pathway will be used to transmit each packet.

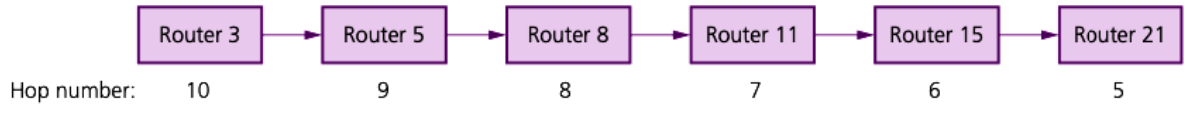


The packets of data start at device A, they could be transmitted down any of the pathways between device A and device B. Each packet can be sent using a different pathway. When a packet reaches a router, the router decides which pathway to send the packet along next. This will continue until all the packets have arrived at device B. It is likely that the packets will arrive out of order. Once all packets have arrived, they are reordered to recreate the file.

Benefits of packet switching	Drawbacks of packet switching
there is no need to tie up a single communication line	packets can be lost and need to be re-sent
it is possible to overcome failed, busy or faulty lines by simply re-routing packets	the method is more prone to errors with real-time streaming (for example, a live sporting event being transmitted over the internet)
it is relatively easy to expand package usage	there is a delay at the destination whilst the packets are being re-ordered.
a high data transmission rate is possible.	

Sometimes it is possible for packets to get lost because they keep 'bouncing' around from router to router and never actually reach their destination. Eventually the network would just grind to a halt as the number of lost packets mount up, clogging up the system. To overcome this, a method called hopping is used. A hop number is added to the header of each packet, and this number is reduced by 1 every time it leaves a router

Each packet has a maximum hop number to start with. Once a hop number reaches zero, and the packet hasn't reached its destination, then the packet is deleted when it reaches the next router. The missing packets will then be flagged by the receiving computer and a request to re-send these packets will be made.

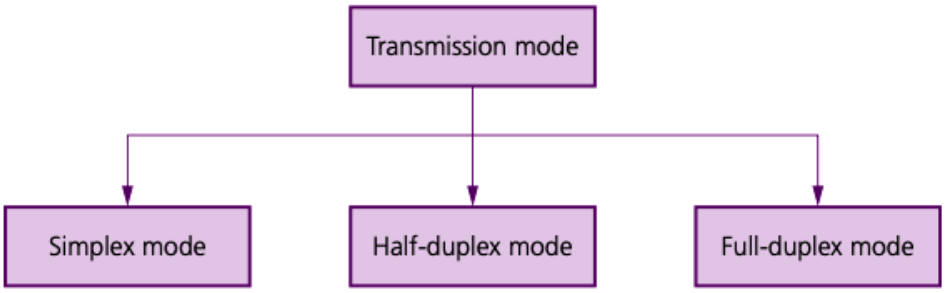


Data transmission

Data transmission can be either over a short distance (for example, computer to printer) or over longer distances (for example, from one computer to another in a global network). Essentially, three factors need to be considered when transmitting data:

- the direction of data transmission (for example, can data transmit in one direction only, or in both directions)
- the method of transmission (for example, how many bits can be sent at the same time)
- how will data be synchronised (that is, how to make sure the received data is in the correct order).

These factors are usually considered by a communication protocol.

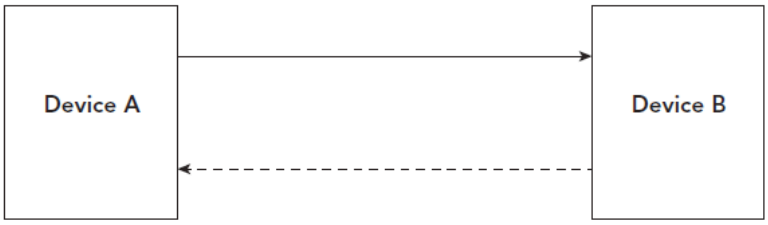


Simplex data transmission

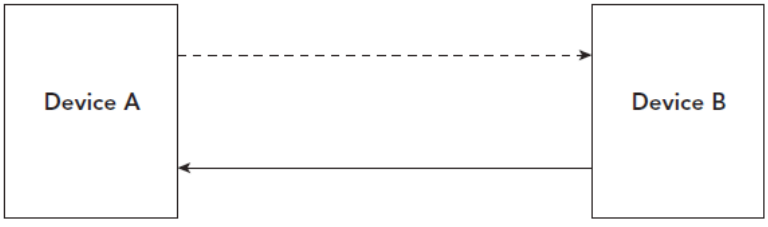
Simplex mode occurs when data can be sent in ONE DIRECTION ONLY (for example, from sender to receiver). An example of this would be sending data from a computer to a printer.

Half-duplex data transmission

Half-duplex mode occurs when data is sent in BOTH DIRECTIONS but NOT AT THE SAME TIME (for example, data can be sent from 'A' to 'B' and from 'B' to 'A' along the same transmission line, but they can't both be done at the same time). An example of this would be a walkie-talkie where a message can be sent in one direction only at a time; but messages can be both received and sent.



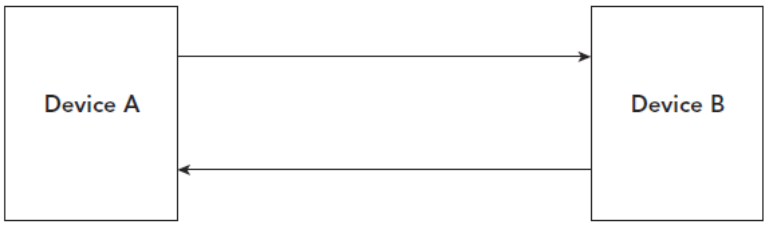
If data is being transmitted from Device A to Device B, then data cannot be transmitted from Device B to Device A, at the same time.

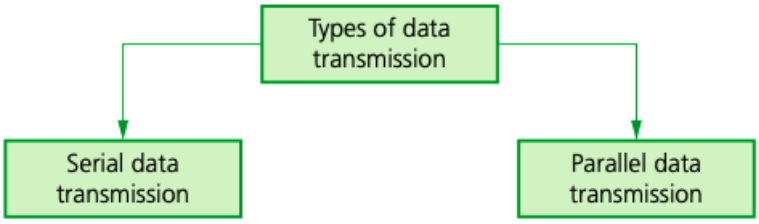


If data is being transmitted from Device B to Device A, then data cannot be transmitted from Device A to Device B, at the same time.

Full-duplex data transmission

Full-duplex mode occurs when data can be sent in BOTH DIRECTIONS AT THE SAME TIME (for example, data can be sent from 'A' to 'B' and from 'B' to 'A' along the same transmission line simultaneously). An example of this would be a broadband internet connection.

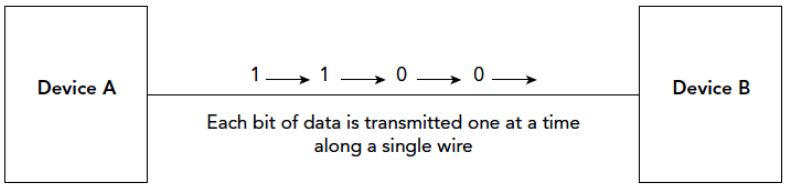




Serial data transmission

Serial data transmission occurs when data is sent ONE BIT AT A TIME over a

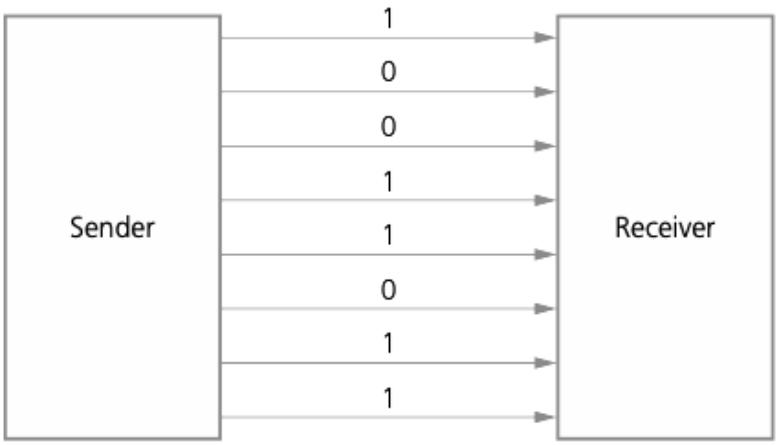
SINGLE WIRE/CHANNEL. Bits are sent one after the other as a single stream.



Advantages	Disadvantages
As data is sent one bit at a time, it should arrive in order of sequence. This means there is less chance of the data being skewed.	As data is sent one bit at a time, the transmission of data is slower.
As data is sent along a single wire, there is less chance of interference. This means there is less chance of error in the data	Additional data may need to be sent to indicate to the receiving device when the data transmission has started and stopped. These are called a start bit and a stop bit
Only one wire is needed for a serial transmission cable, therefore, it is cheaper to manufacture and also cheaper to buy.	

Parallel data transmission

Parallel data transmission occurs when SEVERAL BITS OF DATA (usually one byte) are sent down SEVERAL CHANNELS/WIRES all at the same time. Each channel/wire transmits one bit:



Advantages	Disadvantages
As data is sent multiple bits at the same time, the transmission of data is quicker.	As data is sent multiple bits at the same time, bits do not arrive in order and need to be reordered after transmission. This increases the risk of the data being skewed.
Many computers and devices use parallel data transmission to transmit data internally. Therefore, there is no requirement to convert this to serial data transmission to transmit the data across a network.	As data is sent along multiple wires, there is more chance of interference. This means there is more chance of error in the data.
	Multiple wires are needed for a parallel transmission cable, therefore, it is more expensive to manufacture and also more expensive to buy.

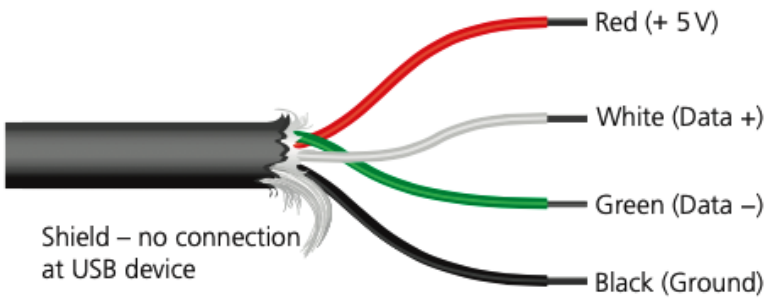
Serial	Parallel
less risk of external interference than with parallel (due to fewer wires)	faster rate of data transmission than serial
more reliable transmission over longer distances	works well over shorter distances (for example, used in internal pathways on computer circuit boards)
transmitted bits won't have the risk of being skewed (that is, out of synchronisation)	since several channels/wires used to transmit data, the bits can arrive out of synchronisation (skewed)
used if the amount of data being sent is relatively small since transmission rate is slower than parallel (for example, USB uses this method of data transmission)	preferred method when speed is important
used to send data over long distances (for example, telephone lines)	if data is time-sensitive, parallel is the most appropriate transmission method
less expensive than parallel due to fewer hardware requirements	parallel ports require more hardware, making them more expensive to implement than serial ports
	easier to program input/output operations when parallel used

Universal serial bus (USB)

As the name suggests, the universal serial bus (USB) is a form of serial data transmission. USB is now the most common type of input/output port found on computers and has led to a standardisation method for the transfer of data between devices and a computer. It is important to note that USB allows both half-duplex and full-duplex data transmission.

the USB cable consists of a four-wired shielded cable, with two wires for power (red and black). The other two wires (white and green) are for data transmission. When a device is plugged into a computer using one of the USB ports:

- the computer automatically detects that a device is present (this is due to a small change in the voltage on the data signal wires in the USB cable)
- the device is automatically recognised, and the appropriate device driver software is loaded up so that the computer and device can communicate effectively
- if a new device is detected, the computer will look for the device driver that matches the device; if this is not available, the user is prompted to download the appropriate driver software (some systems do this automatically and the user will see a notice asking for permission to connect to the device website).



Benefits	Drawbacks
It is a simple interface. The USB cable to device can only fit into the USB one way. Therefore, it means less errors in connecting devices are likely to be made	The length of a USB cable is limited, normally to 5 metres.
The speed of a USB connection is relatively high, so data can be transferred quickly.	
The USB interface is a universal standard for connecting devices, therefore a USB port is included in many different devices.	
When a USB device to cable is inserted into a USB port, it is automatically detected. The first connection will normally involve the download of drivers to operate the hardware that has been connected. Each connection after this the driver should not need to be downloaded again	The transmission speed is relatively high for a USB connection, but it isn't as high as other types of connection, such as ethernet.
A USB connection can also be used to power a device, so it does not need another power source. It can also use this to charge a device, such as a mobile phone.	

Data transmission

Methods of error detection

Zain Merchant

Error detection

Zain Merchant

The transmission of data from one device to another is often not a perfect process. In the process of transmitting the data, interference can occur. This can cause data to be lost, data to be gained and data to change. This would not be helpful and could also lead to many issues. If a password wasn't transmitted correctly, then that person may not be able to log into their account. If a person's address wasn't transmitted correctly, they may not get email that needs to be sent to them. If a person's details are not transmitted correctly to a company, they may not get a product or service that they order. The accuracy of data is often vital, therefore there needs to be a procedure in place to detect any errors in data so that actions can be taken to correct this.

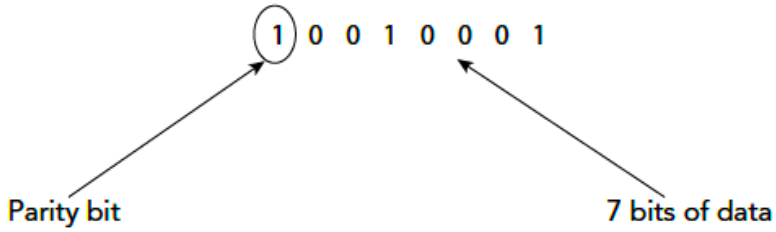
There are several error detection methods that can be used, these include:

- parity check
- checksum
- echo check.

Each of these methods is designed to check for errors after the data has been transmitted from one device to another.

Parity check

A parity check can use an odd or even check method. Each byte of data has 7 bits and 1 extra bit that is called a parity bit. The parity bit is normally the first or last bit of data in the byte.



Before transmission begins, the parity check is set to be either odd or even parity. The number of 1s in the 7 bits of data is totalled. In the example, the result of that would be 2. If an odd parity check is used, then a 1 is added as a parity bit. This is because all the 1s in the byte now add up to 3, which is an odd number. If an even parity check is used a parity bit of 0 would have been added instead. This is because all the 1s in the byte would then add up to 2, which is an even number. When the parity bit has been added to each byte, the data can be transmitted. After transmission, the receiving device will check each byte of data for errors. If an odd parity check has been used and the device finds a byte that has an even number of 1s, then it knows that an error has occurred with this byte of data. The error is detected!

Worked example

In this example, nine bytes of data have been transmitted. Agreement has been made that even parity will be used. Another byte, known as the parity byte, has also been sent. This byte consists entirely of the parity bits produced by the vertical parity check. The parity byte also indicates the end of the block of data.

	Parity bit	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6	Bit 7	Bit 8
Byte 1	1	1	1	1	0	1	1	0
Byte 2	1	0	0	1	0	1	0	1
Byte 3	0	1	1	1	1	1	1	0
Byte 4	1	0	0	0	0	0	1	0
Byte 5	0	1	1	0	1	0	0	1
Byte 6	1	0	0	0	1	0	0	0
Byte 7	1	0	1	0	1	1	1	1
Byte 8	0	0	0	1	1	0	1	0
Byte 9	0	0	0	1	0	0	1	0
Parity byte	1	1	0	1	0	0	0	1

A careful study of table shows the following:

- byte 8 (row 8) now has incorrect parity (there are three 1-bits)
- bit 5 (column 5) also now has incorrect parity (there are five 1-bits).

First of all, the table shows that an error has occurred following data transmission (there has been a change in parity in one of the bytes).

Secondly, at the intersection of row 8 and column 5, the position of the incorrect bit value (which caused the error) can be found. The 1-bit at this intersection should be a 0-bit; this means that byte 8 should have been:

0	0	0	1	0	0	1	0

Checksum

A checksum uses a calculated value to check for errors. A value is calculated from the data that will be transmitted, before transmission takes place. A method such as modulus 11 could be used to calculate the value.

Once the checksum value has been calculated it is added to the data to be transmitted with it. After transmission, the receiving device uses the same method to calculate a value from the received data. If the values match, then the device knows that no error has occurred during transmission. If the values do not match, the device knows that an error has occurred during transmission. The error is detected!

The checksum process is as follows:

- when a block of data is about to be transmitted, the checksum is calculated from the block of data
- the calculation is done using an agreed algorithm (this algorithm has been agreed by sender and receiver)
- the checksum is then transmitted with the block of data
- at the receiving end, the checksum is recalculated by the computer using the block of data (the agreed algorithm is used to find the checksum)
- the re-calculated checksum is then compared to the checksum sent with the data block
- if the two checksums are the same, then no transmission errors have occurred; otherwise a request is made to re-send the block of data.

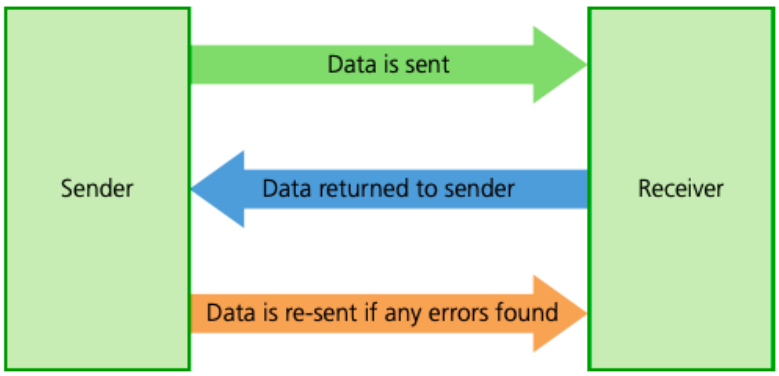
Zain Merchant

Echo check

Zain Merchant

An echo check involves a simple comparison of the data sent to the data received. The sending device transmits the data to the receiving device. The receiving device then transmits the data it receives back to the sending device. The sending device compares the data it sent to the data it has received back from the receiving device to see if they match. If they do, then no error has occurred. If they don't match, then the sending device knows the data was received with error. The error is detected!

- a copy of the data is sent back to the sender
- the returned data is compared with the original data by the sender's computer
- if there are no differences, then the data was sent without error
- if the two sets of data are different, then an error occurred at some stage during the data transmission.



Automatic repeat request (ARQ)

When an error has been detected after the data is transmitted, it is likely that the data will need to be retransmitted. Either the sending device will need to ask the receiving device if it received the data correctly, or the receiving device will need to tell the sending device it did or did not receive the data correctly. This will allow the data to be retransmitted if necessary.

A method called automatic repeat request (ARQ) can be used to do this. There are two main ways that an ARQ can operate and each method uses either a positive or negative acknowledgement and a timeout.

In a positive acknowledgement method:

- The sending device transmits the first data packet.
- The receiving device receives the data and checks it for errors.
- Once the receiving device knows it has received the data error free, it sends a positive acknowledgement back to the sending device.
- When the sending device receives this positive acknowledgement, it knows the receiving device has received the data packet error free and it sends the next data packet.
- If the sending device does not receive a positive acknowledgement within a set timeframe, a timeout occurs.
- When a timeout occurs, the sending device will resend the data packet. It will keep doing this when a timeout occurs, until it receives a positive acknowledgement, or sometimes a limit (such as 20 times) is set and when this limit is reached it will stop resending the data.

In a negative acknowledgement method:

- The sending device transmits the first data packet.
- The receiving device receives the data packet and checks it for errors.
- If the receiving device detects no errors, no further action is taken.
- If the receiving device does detect errors, it will send a negative acknowledgement back to the sender.
- If the sender receives a negative acknowledgement, it knows this means the data was received incorrectly, so it can resend the data packet.
- A timeout is set by the sending device when it sends the data. This is just so that the sending device knows that if it doesn't receive a negative acknowledgement back within that set time period, it doesn't need to be still be waiting for it and can send the next data packet.

Check digit

It is also possible for errors to occur when data entry is performed. This could be manual data entry, for example, a human typing in a value, or it could be automatic data entry, for example, a barcode scanner scanning a barcode to obtain the data stored in the barcode.

A method is necessary to check for errors with data entry and this method is called a check digit. This is how a check digit operates:

- A check digit value is previously calculated from the data that will be entered at some point, for example, a barcode number or an ISBN number. This number is stored with the data.
- When the data is entered, for example, the barcode is scanned, the check digit is recalculated from the data entered.
- If the previously calculated check digit and the stored check digit match, the data entered is correct.
- If the previously calculated check digit and the stored check digit do not match, the data entered is incorrect.

Although the process is similar, you should not confuse the operation of a check digit and a checksum. Make sure that you take the time to understand the difference between the two.

Zain Merchant

Worked example

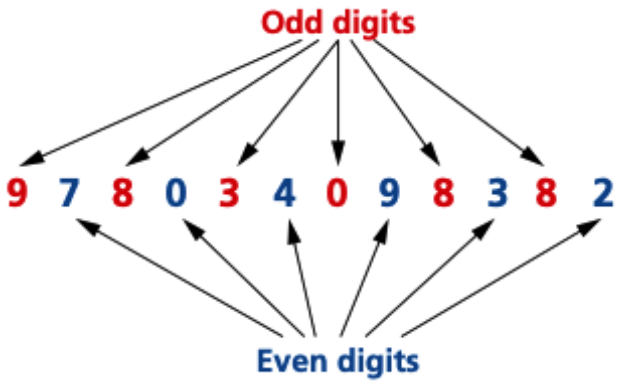
The check digit in ISBN 13 is the thirteenth digit in the number. We will now consider two different calculations. The first calculation is the generation of the check digit. The second calculation is a verification of the check digit (that is, a recalculation).

Calculation 1 – Generation of the check digit from the other 12 digits in a number

The following algorithm generates the check digit from the 12 other digits:

1. add all the odd numbered digits together
2. add all the even numbered digits together and multiply the result by 3
3. add the results from 1 and 2 together and divide by 10
4. take the remainder, if it is zero then use this value, otherwise subtract the remainder from 10 to find the check digit.

1 9 + 8 + 3 + 0 + 8 + 8 = 36
2 3 × (7 + 0 + 4 + 9 + 3 + 2) = 75
3 (36 + 75)/10 = 111/10 = 11 remainder 1
4 10 – 1 = 9 the check digit



Calculation 2 – Re-calculation of the check digit from the thirteen-digit number (which now includes the check digit)

To check that an ISBN 13-digit code is correct, including its check digit, a similar process is followed:

1. add all the odd numbered digits together, including the check digit
2. add all the even number of digits together and multiply the result by 3
3. add the results from 1 and 2 together and divide by 10
4. the number is correct if the remainder is zero.

Using the ISBN 9 7 8 0 3 4 0 9 8 3 8 2 9 (including its check digit) from Figure 2.17:

1. $9 + 8 + 3 + 0 + 8 + 8 + 9 = 45$
2. $3 \times (7 + 0 + 4 + 9 + 3 + 2) = 75$
3. $(45 + 75)/10 = 120/10 = 12$ remainder 0
4. remainder is 0, therefore number is correct.

Data transmission

Encryption

Zain Merchant

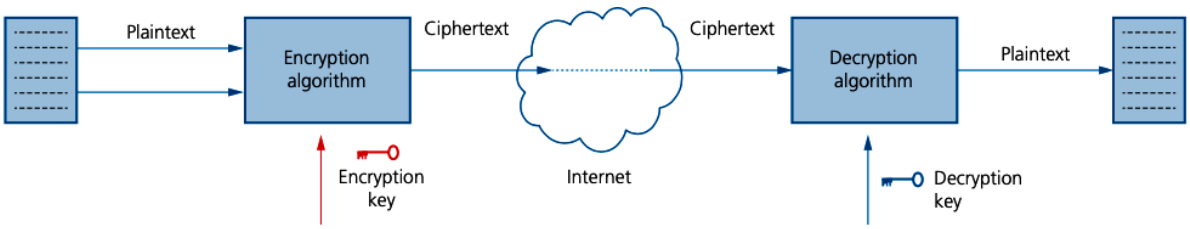
The purpose of encryption

When data is transmitted over any public network (wired or wireless), there is always a risk of it being intercepted by, for example, a hacker. Under these circumstances, a hacker is often referred to as an eavesdropper. Using encryption helps to minimise this risk.

Encryption alters data into a form that is unreadable by anybody for whom the data is not intended. It cannot prevent the data being intercepted, but it stops it from making any sense to the eavesdropper. This is particularly important if the data is sensitive or confidential (for example, credit card/bank details, medical history or legal documents).

Plaintext and cipher text

The original data being sent is known as plaintext. Once it has gone through an encryption algorithm, it produces ciphertext:



Symmetric and asymmetric encryption

Symmetric encryption

Symmetric encryption uses an encryption key; the same key is used to encrypt and decrypt the encoded message. First of all, consider a simple system that uses a 10-digit denary encryption key (this gives 1 × 1010 possible codes); and a decryption key. Suppose our encryption key is:

4 291362856

which means every letter in a word is shifted across the alphabet +4, +2, +9, +1, and so on, places. For example, here is the message COMPUTER SCIENCE IS EXCITING before and after applying the encryption key

C	O	M	P	U	T	E	R
4	2	9	1	3	6	2	8
G	Q	V	Q	X	Z	G	Z

S	C	I	E	N	C	E
5	6	4	2	9	1	3
X	I	M	G	W	D	H

I	S
6	2
O	U

E	X	C	I	T	I	N	G
8	5	6	4	2	9	1	3
M	C	I	M	V	R	O	J

To get back to the original message, it will be necessary to apply the same decryption key; that is, 4 2 9 1 3 6 2 8 5 6. But in this case, the decryption process would be the reverse of encryption and each letter would be shifted –4, –2, –9, –1, and so on. For example, 'G' 'C', 'Q' 'O', 'V' 'M', 'Q' 'P', and so on.

However, modern computers could 'crack' this encryption key in a matter of seconds. To try to combat this, we now use 256-bit binary encryption keys that give 2256 (approximately, 1.2 × 1077) possible combinations. (Even this may not be enough as we head towards quantum computers.)

The real difficulty is keeping the encryption key a secret (for example, it needs to be sent in an email or a text message which can be intercepted). Therefore, the issue of security is always the main drawback of symmetrical encryption, since a single encryption key is required for both sender and recipient.

Asymmetric encryption

Asymmetric encryption was developed to overcome the security problems associated with symmetric encryption. It makes use of two keys called the public key and the private key:

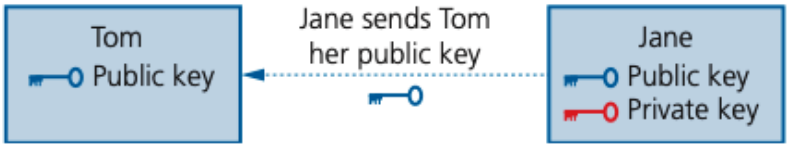
- public key (made available to everybody)
- private key (only known to the computer user).

Both types of key are needed to encrypt and decrypt messages.

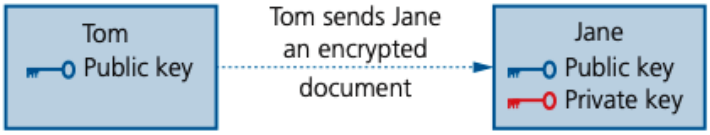
We will use an example to explain how this works; suppose Tom and Jane work for the same company and Tom wishes to send a confidential document to Jane:

1. Jane uses an algorithm to generate a matching pair of keys (private and public) that they must keep stored on their computers; the matching pairs of keys are mathematically linked but can't be derived from each other.

2. Jane now sends her public key to Tom.

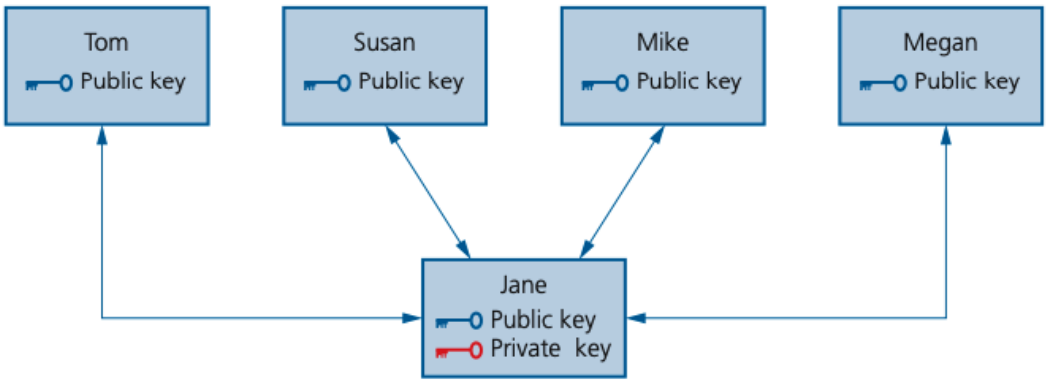


3. Tom now uses Jane's public key ( ) to encrypt the document he wishes to send to her. He then sends his encrypted document (ciphertext) back to Jane.



4. Jane uses her **matching** private key ( ) to unlock Tom's document and decrypt it; this works because the public key used to encrypt the document and the private key used to decrypt it are a matching pair generated on Jane's computer. (Jane can't use the public key to decrypt the message.)

Jane can also exchange her public key with any number of people working in the company, so she is able to receive encrypted messages (which have been encrypted using her public key) and she can then decrypt them using her matching private key:



However, if a two-way communication is required between all five workers, then they all need to generate their own matching public and private keys. Once this is done, all users then need to swap public keys so that they can send encrypted documents/files/messages between each other. Each worker will then use their own private key to decrypt information being sent to them.